

Faculty of Computers and Information Systems

Information Retrieval
Practical Laboratory

Week 2: Regular Expressions for Text Processing

Extract useful values from noisy text before cleaning it. Use the supplied sales paragraph to practice the `re` module and distinguish extraction from replacement.

Duration: approximately 2 practical hours. Suggested pacing: 15 minutes concept review, 25 demonstration, 50 guided implementation, 20 selected exercises, and 10 review. Finish remaining exercises after the session.

1. Lab Objectives

- Write raw-string regex patterns for digits, words, and whitespace.
- Extract numbers and email-like strings with `re.findall()`.
- Remove symbols and normalize whitespace with `re.sub()`.
- Compare character classes and repetition operators on short inputs.

2. Required Background

Prerequisite: basic Python variables, conditions, loops, lists, dictionaries, sets, and functions. Complete the earlier labs before starting this week.

A regular expression describes a pattern of characters. `re.findall(pattern, text)` returns non-overlapping matches. `re.sub(pattern, replacement, text)` returns a modified string. Neither function changes the original string in place.

Use raw strings such as `r"\d+"` to preserve backslashes for the regex engine. A normal string `"\d+"` also works. Some unrecognized Python escapes may currently survive with warnings; a recognized escape such as `"\b"` becomes a backspace. The reliable habit is to use raw patterns.

The slide pattern `r"\S+@\S+"` extracts email-like spans. It is deliberately simple and can include trailing punctuation. Extract emails before removing symbols: deleting `@` and dots destroys the address structure. This exercise is text extraction, not comprehensive email validation.

The slide cleaning pipeline deletes characters outside ASCII letters, digits, and whitespace, then collapses whitespace. Deleting punctuation can join text: `data-base` becomes `database` and email addresses lose punctuation. Week 3 instead extracts letter sequences, which preserves boundaries between separated words.

By default in Python str patterns, `\d` and `\w` are Unicode-aware; `\w` includes letters, digits, and underscore. `[a-zA-Z]` selects ASCII letters explicitly. A dot matches any character except newline unless `DOTALL` is enabled.

Course material: `Regex-in-Python-A-Practical-Guide.pptx`, slides 2-11. The sales paragraph is from slide 11.

3. Python Concepts Used

- **import re:** Load the standard library regex module; no installation is needed.
- **Raw strings:** Use `r"..."` for patterns containing backslashes.
- **Character classes:** `[abc]` means one of a, b, c; `[a-z]` is a range; `[^abc]` excludes the listed characters.
- **Quantifiers:** `+` means one or more, `*` zero or more, `?` zero or one, and `{m,n}` between m and n repetitions.
- **findall and sub:** Return a list of matches or a transformed string.
- **strip and repr:** Remove outer whitespace, or display escape characters visibly.

Pattern	Meaning	Example
\d / \D	Digit / nondigit	\d+ matches 4500
\w / \W	Word character / nonword	\w+ keeps sales_team
\s / \S	Whitespace / nonwhitespace	\s+ includes newlines
[a-zA-Z]+	One or more ASCII letters	Python
colou?r	The u is optional	color and colour
ab*c	Zero or more b characters	ac and abbc
[0-9]{2,4}	Two to four digits	1234

4. Worked Example

This complete example shows the two-stage cleaning pattern from the slides. Extraction happens first, while punctuation and email separators still exist.

```
import re
text = "Hello!! My age is 25@@ years old###"
print(re.findall(r"\d+", text))
text = re.sub(r"^[a-zA-Z0-9\s]", "", text)
text = re.sub(r"\s+", " ", text).strip()
print(text)
```

The first output is ['25']; the final text is Hello My age is 25 years old. Using an empty replacement deletes symbols. Replacing them with a space would preserve more word boundaries. Do not confuse `r"\r\n?"` for normalizing CRLF/CR newlines with `r"\s+"`, which collapses all whitespace into one space.

Supplied sales paragraph

```
Today!! the company sold 15 laptops, 8 phones, and 23 tablets!!!
Manager Ahmed### said profits reached $4500.
Contact us at sales_team@gmail.com or helpdesk@yahoo.com
Thank you for visiting!!!
```

5. Step-by-Step Practical Task

Step 1: Open data/paragraph.txt and read the full four-line sales paragraph. Keep the original text for extraction.

Step 2: Extract all numbers using `r"\d+"` and all email-like spans using `r"\S+@\S+"`.

Step 3: Remove symbols using `r"^[a-zA-Z0-9\s]"` and an empty replacement.

Step 4: Collapse repeated spaces, tabs, and newlines using `r"\s+"`. Strip outer spaces and print the cleaned paragraph.

Step 5: Compare `r"\w+"` with `r"[a-zA-Z]+"` on User_7. Test each quantifier on data/pattern_practice.txt.

Step 6: Finish the starter, returning numbers, emails, and cleaned text. Check all three results independently.

Execution: open a terminal in this week folder and run `python lab02_regex.py`. On Windows, `py -3` can replace `python`. All required files are included; no internet or third-party Python packages are needed.

6. Complete Python Program

The listing below matches lab02_regex.py exactly. Keep indentation when copying. The exercise starter is separate and intentionally unfinished.

lab02_regex.py

```
"""Week 2: extract before cleaning, following the regex slides."""
import os
import re

BASE_DIR = os.path.dirname(os.path.abspath(__file__))

def clean_text(text):
    no_symbols = re.sub(r"^[a-zA-Z0-9\s]", "", text)
    return re.sub(r"\s+", " ", no_symbols).strip()

def main():
    path = os.path.join(BASE_DIR, "data", "paragraph.txt")
    with open(path, "r", encoding="utf-8") as file:
        text = file.read()
    print("Numbers:", re.findall(r"\d+", text))
    print("Emails:", re.findall(r"\S+@\S+", text))
    print("Words:", re.findall(r"[a-zA-Z]+", "Hello, Python 3!"))
    print("Cleaned paragraph:")
    print(clean_text(text))
    # Normalize line endings without collapsing paragraph structure.
    lines = "first\r\nsecond\rthird\n"
    print("Normalized newlines:", repr(re.sub(r"\r\n?", "\n", lines)))

if __name__ == "__main__":
    main()
```

7. Expected Output

Representative output from the shipped collection and the default command. Calculated values come from a tested program run. Long output lines wrap here for printing; expected_output.txt contains the full terminal output. Selected rows are shown for large matrices.

```
Numbers: ['15', '8', '23', '4500']
Emails: ['sales_team@gmail.com', 'helpdesk@yahoo.com']
Words: ['Hello', 'Python']
Cleaned paragraph:
Today the company sold 15 laptops 8 phones and 23 tablets Manager Ahmed said profits
reached 4500 Contact us at salesteamgmailcom or helpdeskyaahoocom Thank you for
visiting
Normalized newlines: 'first\nsecond\nthird\n'
```

8. Laboratory Exercises

1. Basic: compare `r"\d"` and `r"\d+"` on Order 1234 costs \$56. Explain why the lists differ.
2. Implementation: extract words using `r"[a-zA-Z]+"` and email-like strings from the supplied paragraph.
3. Implementation: remove punctuation `[!?.]` from a short sentence, then remove all unwanted symbols from the sales paragraph.
4. Debugging: explain what is wrong with `r"[a-z | A-Z]"` when the intention is to match letters. Correct it and add `+` to extract whole sequences.
5. Interpretation: normalize CRLF and CR to LF while retaining line breaks; separately collapse all whitespace. Compare the outputs.
6. Extension: show how the simple email pattern includes a trailing comma. Try `r"[\w.+]+@[ \w.-]+\.[A-Za-z]{2,}"` on the practice file and discuss its remaining limits.

7. Quantifiers: test +, *, ?, and {2,4}; include an input that produces an empty match with *. Explain that zero occurrences are allowed.

9. Student Tasks

- A completed starter and the unchanged supplied paragraph.
- Outputs for numbers, emails, and the cleaned paragraph, plus your newline-normalization experiment.
- A short explanation of why extraction must precede destructive cleaning.

Submit your own implementation and evidence. Do not submit an untouched starter as a completed solution. Label all outputs with the query or input that produced them.

10. Review Questions

1. How do the return values of findall and sub differ?
2. Why are raw strings useful in regex patterns?
3. Does \w mean letters only?
4. Why can deleting punctuation join adjacent text?
5. What is the difference between + and *?

11. Challenge Exercise

Optional: Write a cleaning function with a keep_newlines option. Preserve LF line boundaries while collapsing repeated spaces and tabs, then test it on mixed Windows and Unix line endings.